

// Jivops

Cassandra

Guía completa — 20 lecciones

Fundamentos y modelado · Replicación y consistencia · Operación y
mantenimiento · Seguridad y producción

Contenido

Módulo 1: Fundamentos y modelado de datos

- Lección 01 — Qué es Cassandra y su arquitectura sin maestro
- Lección 02 — El anillo de tokens, el partitioner y los vnodes
- Lección 03 — Modelo de datos — partition key, clustering y diseño por consulta
- Lección 04 — Particiones grandes y hot partitions
- Lección 05 — CQL y sus restricciones — ALLOW FILTERING

Módulo 2: Replicación, consistencia y rutas de lectura y escritura

- Lección 06 — Factor de replicación, estrategias y snitch
- Lección 07 — Niveles de consistencia y quórum
- Lección 08 — La ruta de escritura — commit log, memtable y SSTables
- Lección 09 — La ruta de lectura y los bloom filters
- Lección 10 — Tombstones, gc_grace_seconds y datos zombi

Módulo 3: Operación y mantenimiento

- Lección 11 — Estrategias de compactación — STCS, LCS y TWCS
- Lección 12 — Repair y anti-entropía
- Lección 13 — Gossip, detección de fallos y hinted handoff
- Lección 14 — Añadir y retirar nodos — bootstrap, cleanup y decommission
- Lección 15 — Backups y snapshots

Módulo 4: Rendimiento, seguridad y producción

- Lección 16 — Batches y transacciones ligeras (LWT)
- Lección 17 — Dimensionamiento, hardware y espacio libre
- Lección 18 — Seguridad — autenticación, roles, TLS y system_auth
- Lección 19 — Monitoreo — nodetool, tpstats y métricas clave
- Lección 20 — Proyecto final: Cassandra lista para producción

Módulo 1: Fundamentos y modelado de datos

Lección 01 — Qué es Cassandra y su arquitectura sin maestro

Es una base de datos distribuida y sin nodo maestro (todos los nodos cumplen el mismo papel), orientada a columnas anchas y optimizada para escrituras, que escala añadiendo nodos, replica entre centros de datos y permite elegir el nivel de consistencia por consulta.

No existe un punto único de fallo porque cualquier nodo puede atender peticiones; añadir nodos aumenta la capacidad sin cuellos de botella en un coordinador central, y la caída de un nodo no detiene el cluster.

Lección 02 — El anillo de tokens, el partitioner y los vnodes

Los datos se reparten por un anillo de tokens: el partitioner (Murmur3 por defecto) calcula un token a partir de la partition key y determina qué nodos son responsables; los vnodes dan a cada nodo varios rangos, el factor de replicación indica cuántas copias hay y cualquier nodo puede actuar como coordinador de una petición.

El hash distribuye las claves de forma pseudoaleatoria por todo el anillo, así que claves parecidas no acaban en el mismo nodo y se evita concentrar datos o tráfico en pocos nodos.

Lección 03 — Modelo de datos — partition key, clustering y diseño por consulta

La clave primaria se compone de la partition key (decide en qué nodos vive el dato) y de columnas de clustering (ordenan las filas dentro de la partición); se modela a partir de las consultas, creando una tabla por cada patrón de acceso y desnormalizando, en lugar de normalizar y unir con joins.

Cada consulta eficiente necesita leer una o pocas particiones con la clave adecuada, y no hay joins ni filtrado eficiente por cualquier columna; el almacenamiento y las escrituras son baratos, así que se prefiere duplicar a unir.

Lección 04 — Particiones grandes y hot partitions

Una partición que crece sin límite (por ejemplo, todos los eventos de un usuario para siempre) se vuelve lenta y pesada, y una partition key con pocos valores o muy popular crea una hot partition que satura sus réplicas; se mitiga añadiendo un bucket a la clave (por ejemplo usuario + día) para acotar el tamaño y repartir la carga.

Cuanto más crece, más cuesta leerla, compactarla, repararla y moverla entre nodos, y las consultas sobre ella tardan más y consumen más memoria; acotar el tamaño con buckets mantiene las operaciones predecibles.

Lección 05 — CQL y sus restricciones — ALLOW FILTERING

CQL parece SQL pero no tiene joins ni subconsultas, y el WHERE debe indicar la partition key (con igualdad) y, en orden, las columnas de clustering; ALLOW FILTERING fuerza recorridos que pueden barrer muchas particiones y debe evitarse, y los índices secundarios solo sirven en casos concretos.

No se sabe en qué nodos está el dato y hay que consultar muchos o todos los nodos, así que el coste crece con el tamaño del cluster; las consultas deben apuntar a una partición concreta.

Módulo 2: Replicación, consistencia y rutas de lectura y escritura

Lección 06 — Factor de replicación, estrategias y snitch

El factor de replicación (RF) define cuántas copias de cada dato existen; NetworkTopologyStrategy fija el RF por centro de datos y, con un snitch que informa de DC y rack, reparte las réplicas en racks distintos, mientras que SimpleStrategy ignora la topología y no es apropiada para producción multi-DC.

Un fallo de rack no elimina todas las copias de un dato, ya que las réplicas en el mismo rack comparten alimentación y red; con racks distintos el dato sigue disponible.

Lección 07 — Niveles de consistencia y quórum

Indican cuántas réplicas deben responder: con RF 3, QUORUM son 2 y si lecturas y escrituras usan QUORUM ($R + W > RF$) se lee siempre lo último escrito; LOCAL_QUORUM evita la latencia entre centros de datos, ONE es más rápido pero puede devolver datos desactualizados y ALL falla si cae cualquier réplica.

Los conjuntos de réplicas de lectura y escritura siempre se solapan en al menos una réplica, y esa réplica común tiene la última escritura; por eso se usa QUORUM en lecturas y escrituras.

Lección 08 — La ruta de escritura — commit log, memtable y SSTables

Se anexa al commit log (durabilidad, secuencial) y se aplica a la memtable en memoria; cuando esta se llena se vuelca como SSTable inmutable y la compactación fusiona SSTables; no hay actualización en sitio, así que las escrituras son rápidas y, ante versiones distintas de una celda, gana la de mayor timestamp, por lo que los relojes deben estar sincronizados.

La versión con mayor timestamp gana, así que un reloj desfasado hace que una escritura antigua pise a una más reciente; los timestamps los asigna el coordinador o el cliente y un desfase puede provocar pérdida aparente de escrituras.

Lección 09 — La ruta de lectura y los bloom filters

Combina la memtable y las SSTables que contengan la partición, usando bloom filters para descartar las que no la tienen e índices para localizarla, y fusiona los resultados por timestamp; cuantas más SSTables haya que consultar, más lenta es la lectura, de ahí la importancia de la compactación.

Hay que consultar y fusionar los fragmentos de la partición de cada SSTable, lo que supone más accesos a disco y más trabajo por lectura; la compactación reduce el número de SSTables.

Lección 10 — Tombstones, gc_grace_seconds y datos zombi

Borrar no elimina el dato al instante: escribe una tombstone que se conserva al menos gc_grace_seconds (10 días por defecto) para que llegue a todas las réplicas; si una réplica estuvo caída más tiempo y no se ejecutó repair, el dato borrado puede reaparecer; las tombstones acumuladas ralentizan las lecturas, y los patrones de cola (insertar y borrar) o los TTL masivos las generan en abundancia.

Cada borrado genera una tombstone y las lecturas deben recorrerlas hasta que se purguen, así que se acumulan y ralentizan o hacen fallar las consultas; Cassandra está pensada para escrituras que se acumulan, no para colas.

Módulo 3: Operación y mantenimiento

Lección 11 — Estrategias de compactación — STCS, LCS y TWCS

STCS (por defecto) fusiona SSTables de tamaño similar y favorece cargas con muchas escrituras, aunque puede necesitar mucho espacio libre; LCS organiza niveles con pocas SSTables por lectura y conviene a cargas con muchas lecturas y actualizaciones, a costa de más E/S; TWCS agrupa los datos por ventanas de tiempo y encaja con series temporales con TTL.

Los datos antiguos ya están en ventanas cerradas y actualizarlos los reparte en muchas SSTables, perdiéndose la ventaja de no recompartar ventanas viejas; la estrategia asume que los datos no cambian después de su ventana.

Lección 12 — Repair y anti-entropía

Compara réplicas con árboles de Merkle y sincroniza las diferencias; hay que ejecutarlo de forma regular en cada nodo dentro de `gc_grace_seconds` (por ejemplo programado con Cassandra Reaper), porque ni los hints ni el read repair garantizan que todas las réplicas converjan.

Asegura que todas las réplicas reciban las tombstones antes de que se purguen; si una réplica no recibió el borrado y la tombstone se purga, el dato antiguo reaparece, por eso debe completarse dentro de `gc_grace_seconds`.

Lección 13 — Gossip, detección de fallos y hinted handoff

Los nodos intercambian su estado por gossip y un detector de fallos decide cuándo marcar un nodo como caído; mientras una réplica está caída, el coordinador guarda hints y los reenvía cuando vuelve, pero solo durante la ventana configurada (3 horas por defecto): si el nodo tarda más, hace falta repair.

Los hints se guardan solo durante una ventana limitada y pueden perderse o descartarse; pasada la ventana, la réplica queda sin los cambios, y el repair sincroniza todo lo que falta.

Lección 14 — Añadir y retirar nodos — bootstrap, cleanup y decommission

Al añadir un nodo, este hace bootstrap: asume rangos de tokens y recibe los datos por streaming; después hay que ejecutar `nodetool cleanup` en los nodos antiguos para eliminar los datos que ya no les corresponden; para retirar un nodo vivo se usa `decommission` (transfiere sus datos) y `removenode` para uno muerto, haciéndolo de uno en uno.

Los nodos antiguos conservan datos de rangos que ahora pertenecen al nodo nuevo, y esos datos sobrantes ocupan espacio hasta que se eliminan; `cleanup` borra lo que ya no les corresponde.

Lección 15 — Backups y snapshots

`nodetool snapshot` crea enlaces duros a las SSTables de forma casi instantánea en cada nodo, y las copias deben guardarse fuera del nodo; con incremental backups y archivado del commit log se logra recuperación a un punto en el tiempo; la replicación no es un backup, porque un borrado o una corrupción lógica se replican a todas las réplicas.

Los snapshots son enlaces a los mismos archivos y se pierden con el disco o el nodo; una copia debe guardarse fuera del nodo para sobrevivir a su pérdida, y además hay que probar la restauración.

Módulo 4: Rendimiento, seguridad y producción

Lección 16 — Batches y transacciones ligeras (LWT)

Un logged batch garantiza que las escrituras (típicamente a varias tablas desnormalizadas) se apliquen todas o ninguna, pero cuesta más y no sirve para cargas masivas, y un batch con muchas particiones sobrecarga al coordinador; las transacciones ligeras (IF NOT EXISTS) usan Paxos y varias rondas de red, así que se reservan para casos concretos.

El coordinador debe gestionar y reenviar escrituras a muchos nodos distintos y se sobrecarga; el batch está pensado para atomicidad, no para rendimiento, y es mejor enviar escrituras individuales asíncronas a su partición.

Lección 17 — Dimensionamiento, hardware y espacio libre

Discos locales SSD o NVMe (no SAN ni NAS), un heap de JVM moderado dejando el resto de la RAM a la caché de archivos del sistema, y mantener espacio libre en disco para la compactación: con STCS puede necesitarse hasta alrededor de la mitad del volumen libre.

La compactación escribe nuevas SSTables antes de borrar las antiguas y necesita espacio libre; sin él no puede compactar, se acumulan SSTables que ralentizan las lecturas y puede llegar a impedirse escribir.

Lección 18 — Seguridad — autenticación, roles, TLS y system_auth

Por defecto Cassandra no exige autenticación (AllowAllAuthenticator): hay que activar PasswordAuthenticator y un autorizador con roles, cambiar la cuenta cassandra/cassandra por defecto, replicar el keyspace system_auth en varios nodos, cifrar con TLS el tráfico entre clientes y nodos y entre nodos, y no exponer los puertos de CQL, internodo ni JMX a redes no confiables.

Las credenciales y roles viven solo en un nodo y los demás no pueden validar a los usuarios; si ese nodo cae, nadie puede autenticarse, por eso se aumenta el factor de replicación de system_auth.

Lección 19 — Monitoreo — nodetool, tpstats y métricas clave

nodetool status muestra el estado de cada nodo (UN = arriba y normal); tpstats revela colas pendientes, bloqueadas y mensajes descartados (dropped), señal de sobrecarga; hay que vigilar latencias por percentil, compactaciones pendientes, tombstones leídas, hints, uso de disco y pausas de GC, normalmente con un exporter a Prometheus.

La media oculta las peticiones lentas, que son las que sufren los usuarios; un p99 alto indica problemas como compactación, tombstones o GC, y en sistemas distribuidos las colas largas dominan la experiencia.

Lección 20 — Proyecto final: Cassandra lista para producción

Tablas diseñadas por consulta con particiones acotadas, RF 3 por DC con NetworkTopologyStrategy y consistencia LOCAL_QUORUM, compactación adecuada al patrón de uso, repair regular dentro de gc_grace, backups fuera del nodo y probados, autenticación, TLS y system_auth replicado, y monitoreo de latencias, tombstones y compactaciones.

Intenta mencionar al menos dos pilares: tablas diseñadas por consulta con particiones acotadas, RF 3 por DC con LOCAL_QUORUM, compactación y repair regular, backups fuera del nodo y probados, o autenticación, TLS y monitoreo.